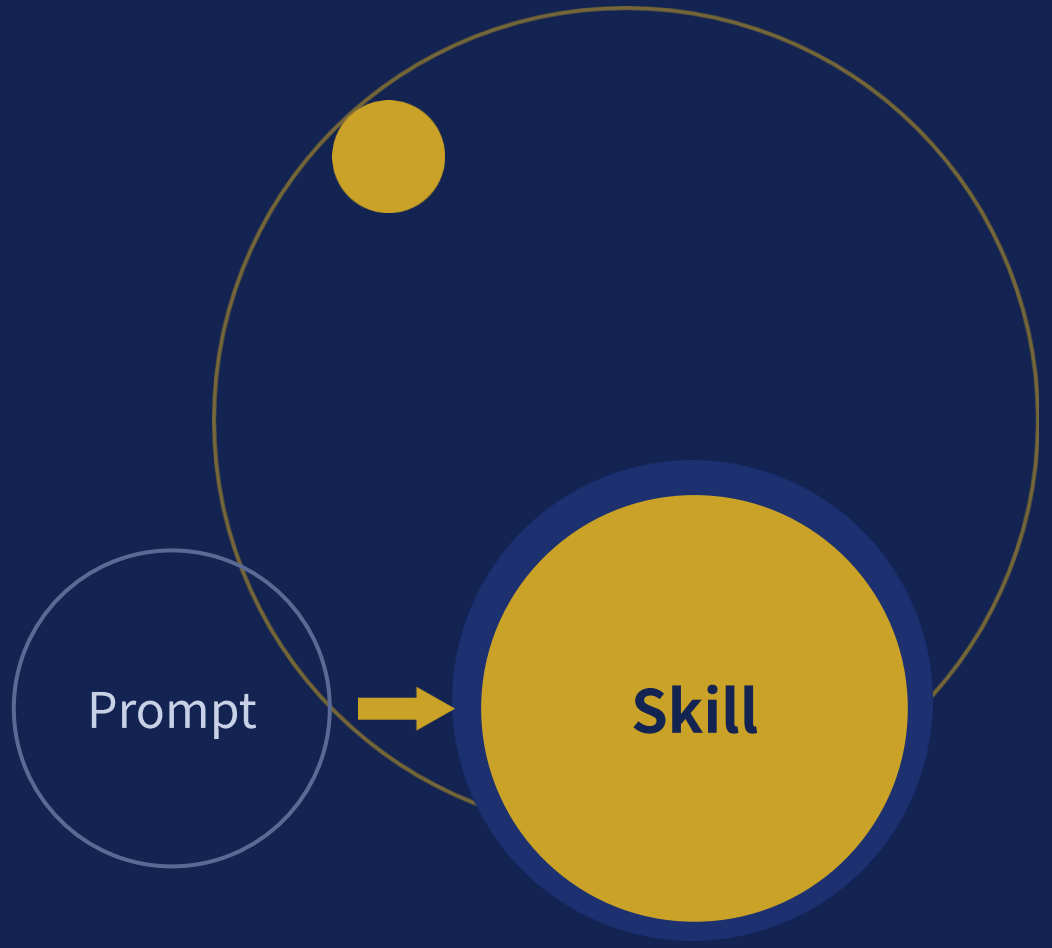
2026／10／14（三）20:00–22:00 ｜ 線上研討會

掌握 Agent Skills
打造專屬 AI 技能包

王孝弘
wang@ispan.com.tw

上課範例：
https://github.com/HsiaoHungWang/skill-demo-1014



資展國際 iSpan International

一般

好提示詞的七個元素：也是 Skill 的原料

1 角色

你是誰

例：你是資深行政助理

2 任務

要做什麼

例：整理成會議紀錄

3 背景

給誰看、為什麼

例：主管週一開會前要讀

4 步驟

怎麼做

例：先寫摘要，再列待辦

5 輸出格式

長什麼樣

例：表格：事項／負責人／期限

6 範例

給一個樣本

例：附上一份上週的紀錄

7 限制

哪些不能做

例：不確定就標「待確認」

七項寫齊＝SKILL.md 內文

再補上 name＋description，就是一個 Skill

寫法原則：說清楚、說明原因、說「要做什麼」，詳見第 6 頁。

資料來源：Claude Docs〈Prompting best practices〉

一般

七元素範例：會議紀錄整理（行政）

【角色】你是一位資深行政助理，熟悉本公司的會議流程。

【任務】請把下面的會議逐字稿整理成會議紀錄與待辦清單。

【背景】這份紀錄會在週一早上寄給部門主管，主管通常只有 3 分鐘閱讀時間，所以重點要一眼看得到。

【步驟】

1. 先用 3 句話寫會議摘要
2. 再列出所有決議事項
3. 最後整理待辦事項

【輸出格式】待辦事項用表格呈現，欄位為：事項／負責人／期限。

【範例】請參考上週的紀錄格式：

｜更新招生簡章｜王小明｜10/20｜

【限制】逐字稿中沒有明確指定負責人或期限時，請填「待確認」，因為猜錯的名字會讓同事白做工。

一般

七元素範例：系統異常事件報告（工程師）

【角色】你是一位資深維運工程師，熟悉事件檢討流程。

【任務】根據下方的錯誤日誌與處理紀錄，整理一份事件報告。

【背景】報告要給非技術背景的部門主管看，他們只關心三件事：影響範圍、是否已解決、會不會再發生。

【步驟】

1. 先用一句話說明發生了什麼事
2. 列出時間軸（發現 → 處理 → 恢復）
3. 說明根本原因與預防措施

【輸出格式】一句話摘要，加上時間軸表格（時間｜事件｜處理人），再列 3 點預防措施，全文 300 字以內。

【範例】「10/3 14:05 會員登入服務中斷 22 分鐘，約 1,200 位使用者受影響，已修復並加入監控。」

【限制】不出現程式碼和英文縮寫。日誌看不出原因的地方寫「待確認」，不可自行推測。

一般

七元素範例：月營運數據摘要（主管）

【角色】你是一位營運分析師，負責每月向管理層報告。

【任務】根據附上的本月與上月營運數據，寫一份主管摘要。

【背景】讀者是總經理，只有 2 分鐘可以看，要能直接據此決定下個月的資源配置。

【步驟】

1. 找出與上月相比變化最大的 3 項指標
2. 每項說明可能原因
3. 提出 1 個需要決策的事項

【輸出格式】3 張重點卡（指標｜本月｜與上月比較｜原因），加上一段「需要您決定」，200 字以內。

【範例】「不良率 2.8%（↑0.9 個百分點），主因是新供應商來料，建議本月啟動來料抽驗。」

【限制】只使用表格中的數字，不自行估算。沒有資料支撐的原因要標註「推測」。

一般

提示詞的三個寫法原則

1 說清楚、說具體

AI 不會猜你心裡的標準；新同事看不懂的指令，AI 也看不懂

原本寫法

幫我整理會議紀錄。

改寫後

整理成 3 句摘要，加上待辦表格（事項／負責人／期限）。

2 說明原因

AI 知道「為什麼」，遇到沒寫到的類似狀況，也能自己判斷怎麼處理

原本寫法

不確定負責人就填「待確認」。

改寫後

不確定負責人就填「待確認」，因為猜錯的名字會讓同事白做工。

3 說「要做什麼」

正面指令比禁止句更容易照做，AI 不必自己猜替代方案

原本寫法

不要用太多專業術語。

改寫後

用新進同事也看得懂的說法，術語第一次出現時附上解釋。

黃金法則：寫給一個不了解背景的同事看。寫進 SKILL.md 一次，每次觸發都自動生效。

資料來源：Claude Docs〈Prompting best practices〉

一般

模型變強之後，七個元素還重要嗎？

元素	現在的重要性	原因
3 背景	▲ 更重要	模型再強，也不知道讀者是誰、為什麼要做
7 限制	▲ 更重要	新模型很會照字面執行，沒講的邊界會自己決定
5 輸出格式	● 依然重要	要接下一個流程時，格式就是規格
2 任務	● 依然重要	一句話講清楚要什麼，永遠是基本功
6 範例	● 重要但要小心	效果很強，但模型會高度模仿，給一個就全長一樣
4 步驟	▼ 變弱	推理型模型會自己規劃，只需寫公司規定的流程
1 角色	▼ 變弱	「你是資深 XX」效果有限，背後的背景才是關鍵

趨勢

Prompt Engineering

怎麼措辭，模型才聽得懂？

Context Engineering

要給模型哪些資訊，它才做得對？

Skill 就是 Context Engineering 的落地形式：

背景放 references、格式放 assets、限制寫進 SKILL.md，需要時才載入。

七要素是檢查清單，不是公式。模型越強，你越不需要教它怎麼想，但越需要告訴它你的世界。

參考：Claude Docs〈Prompt engineering〉

Prompt 使用的五大痛點

1	**每次都要重複輸入** 時間黑洞	同一套要求每次重貼，漏掉一項輸出就不對
2	**輸出品質不穩定** 時好時壞	結果取決於當天 Prompt 寫得有多完整
3	**知識無法累積** 每次從零開始	好 Prompt 埋在聊天記錄裡，人走了知識也帶走
4	**上下文爆炸** 塞太多反而分心	參考資料每次全貼，耗 token 又容易忘記要求
5	**多任務無法協作** 單打獨鬥	任務環環相扣，中間結果只能手動複製貼上

為了解決這個問題， Anthropic 推出了 Agent Skills，並在 2025 年 12 月開放為業界標準，ChatGPT、Codex、Gemini.... 都已支援。

一句話定義

Agent Skills 是「可重複使用的 AI 知識包」

把這些東西打包在一起：

Prompt／流程
SKILL.md

＋

參考文件
references/

＋

範本
assets/

＋

程式碼
scripts/

讓 AI 在對的時機，自動使用對的知識

每個職位，都有自己的 Skill

工程師／IT

程式碼審查 Skill

貼上程式碼變更，依團隊規範檢查命名、例外處理與資安風險，列出修改建議與嚴重程度。

主管／管理職

部門週報彙整 Skill

收齊成員週報，彙整成進度、風險、需要決策三段的一頁摘要，2 分鐘看完。

會計／財務

費用報銷檢核 Skill

上傳發票與報銷單，對照公司報支規範逐項檢查，標出超額、缺件與異常項目。

品管／製造

異常報告 Skill

貼上客訴或異常紀錄，依公司格式整理成原因分析、暫時對策與預防措施，未確認處標「待確認」。

行政／總務

會議紀錄整理 Skill

丟入會議逐字稿，產出 3–5 點決議摘要＋待辦表格，負責人與時限一目了然。

教育訓練

課程教案產生 Skill

給定課程主題與時數，依公司教案格式產出大綱、課堂活動與課後測驗題。

判斷口訣：只要是「固定會做、資料固定、步驟講得清楚、成果能檢查、出錯有人把關」的工作，就值得寫成一個 Skill。

知識無法累積 — 每次從零開始

純 Prompt 的命運

- 存在聊天記錄 #47 裡，下次要找 10 分鐘
- 同事問怎麼寫，截圖給他，貼上後格式跑掉
- 三個月後忘了當初怎麼寫，重新調一次
- 新人進來，每個人都要重新摸索

SKILL 是一個檔案，可以…

Git 版本控制
commit・diff 看得到每次修改

公司共用雲端
Google Drive・SharePoint

打包 zip 分享
一個檔案傳給同事

新人直接可用
下載即用，不用重新摸索

☆ **SKILL 讓「個人經驗」變成「團隊資產」——人離開了，知識留得住**

上下文爆炸 — 塞太多反而分心

純 Prompt 做法

「請分析廣告數據。對了，ROAS 是收入／花費，大於 4 叫優秀…台灣電商平均 CTR 是 1.2%…輸出請用這個格式：**[貼上 50 行 HTML 模板]**…」

- 每次貼上所有參考資料，佔用大量 token
- AI 容易分心，忘了前面的要求
- 資料一更新，所有 Prompt 都要改

SKILL 解法：拆成資料夾

meta-ads-analyzer/

```
├── SKILL.md
|   ↑ 只放流程與框架（精簡）
├── references/
|   ├── metrics-guide.md
|   |   ↑ 指標定義
|   └── industry-benchmarks.md
|       ↑ 行業基準
└── assets/
    └── report-template.html
        ↑ 報告模板
```

Progressive Disclosure（漸進式揭露）

1

第 1 層 ・ 永遠載入

description

~100 tokens

Agent 先看這裡，決定「要不要用我」

2

第 2 層 ・ 觸發後載入

SKILL.md 全文

< 5,000 tokens

讀完整流程，決定「怎麼執行」

3

第 3 層 ・ 需要時才讀

references / assets

按需載入

用到哪份資料，才讀哪份

不用一次塞爆：平常只佔約 100 tokens，真正需要時才逐層展開

多任務無法協作 — 單打獨鬥

純 Prompt：三個獨立對話，中間結果手動貼

對話 1
分析廣告
得到分析結果

→

對話 2
規劃預算
＋手動貼上分析結果

→

對話 3
優化文案
＋手動貼上預算結果

SKILL：一句話自動串接

「分析這週廣告，規劃下週預算，優化最好的文案」

載入並分析
meta-ads-analyzer

→

需要「預算」
budget-planner

→

需要「文案」
ad-copy-optimizer

→

整合輸出
完整報告

Prompt vs Agent Skill

維度	純 Prompt	Agent Skill
本質	一次性對話指令	可重複呼叫的知識模組
存放位置	聊天記錄／記事本	獨立檔案，可版本控制
觸發方式	手動複製貼上	依 description 自動判斷觸發
輸出穩定性	看當天寫得好不好	固定格式，輸出一致
知識範圍	只有 Prompt 本身	Prompt＋參考資料＋範本＋程式碼
團隊共享	截圖／複製／口頭傳授	打包 zip，下載即用
可維護性	改一處，所有副本都要改	改一份 SKILL.md，統一更新
複雜度上限	受限於上下文長度	可擴充 references／scripts，按需載入

Prompt 是「一次性對話」，Skill 是「可重複使用的知識包」

什麼時候該用 Skill？

如果你…	純 Prompt 就夠了	你需要 SKILL
偶爾用一次	✓ 適用	—
每天重複做同一件事	—	✓ 適用
需要輸出品質穩定	—	✓ 適用
要和同事共享最佳實踐	—	✓ 適用
任務複雜，需要參考很多資料	—	✓ 適用
多個任務需要串接協作	—	✓ 適用

SKILL 不是取代 Prompt，而是把「調好的 Prompt」封裝成「可重複使用的工具」

一般

一次撰寫，多個平台都能用

Claude 網頁版

Customize › Skills › Upload skill

上傳資料夾壓縮的 .zip

Gemini 網頁版

設定 › 技能 › 上傳

只收單一 SKILL.md 檔案



ChatGPT 免費版

附上 SKILL.md › @Plugin Creator

自動打包成私人外掛

從平台下載回來

Claude、Gemini 下載即標準格式

可再上傳到其他平台

最大公約數：只用一個 SKILL.md，三個平台都能匯入；附加資料夾只有 Claude、ChatGPT 支援

資料來源：Claude、ChatGPT、Gemini 免費版實測（2026／09／30–10／01）；agentskills.io 開放標準

在各平台使用 Skill ｜ 瀏覽器裡的 AI

Chrome 側邊欄 Gemini：Skills 三步驟

1 建立

側邊欄試好提示詞 › ⋯ › 儲存為技能

或從「技能庫」套用官方範本

2 管理

在技能清單編輯、改名、刪除

綁定 Google 帳號，跨裝置同步

3 使用

輸入框打 / › 選擇技能

可勾選多個分頁一次執行（最多 10 個）

和 SKILL.md 技能包的差別

Chrome Skills 是「存起來的提示詞」，是從下指令走向技能包的入門版



一般

Gemini 免費版：對話直接建立 Skills

https://gemini.google.com/app

1 建立

設定 › 技能 › 用 Gemini 建立

或對話說「幫我建立…技能」

2 管理

建立後自動存檔並啟用

上傳只收單一 SKILL.md，不收 ZIP

3 使用

輸入 /技能名稱，再貼上內容

實測：只說「整理會議記錄」未觸發



一般

ChatGPT 免費版：用 Plugin Creator 做 Skills

1 建立

外掛程式 › ＋ › 建立外掛程式

@Plugin Creator ＋ 一句話描述需求

2 修改

對 @Plugin Creator 說明要改哪裡

版本自動遞增，改完務必開新對話

3 使用

直接說「整理會議記錄」即自動觸發

或用 @技能名稱 明確指定



ZIP 格式

ChatGPT 外掛程式要求 zip 裡的 SKILL.md 放在 skills/<名稱>/SKILL.md

一般

Claude 免費版：用對話建立 Skills

1 建立

Settings > Customize › Skills › ＋ Add

或對話說「幫我建立…skill」

2 修改

對話說明要改哪裡，再存檔

改完一樣要開新對話測試

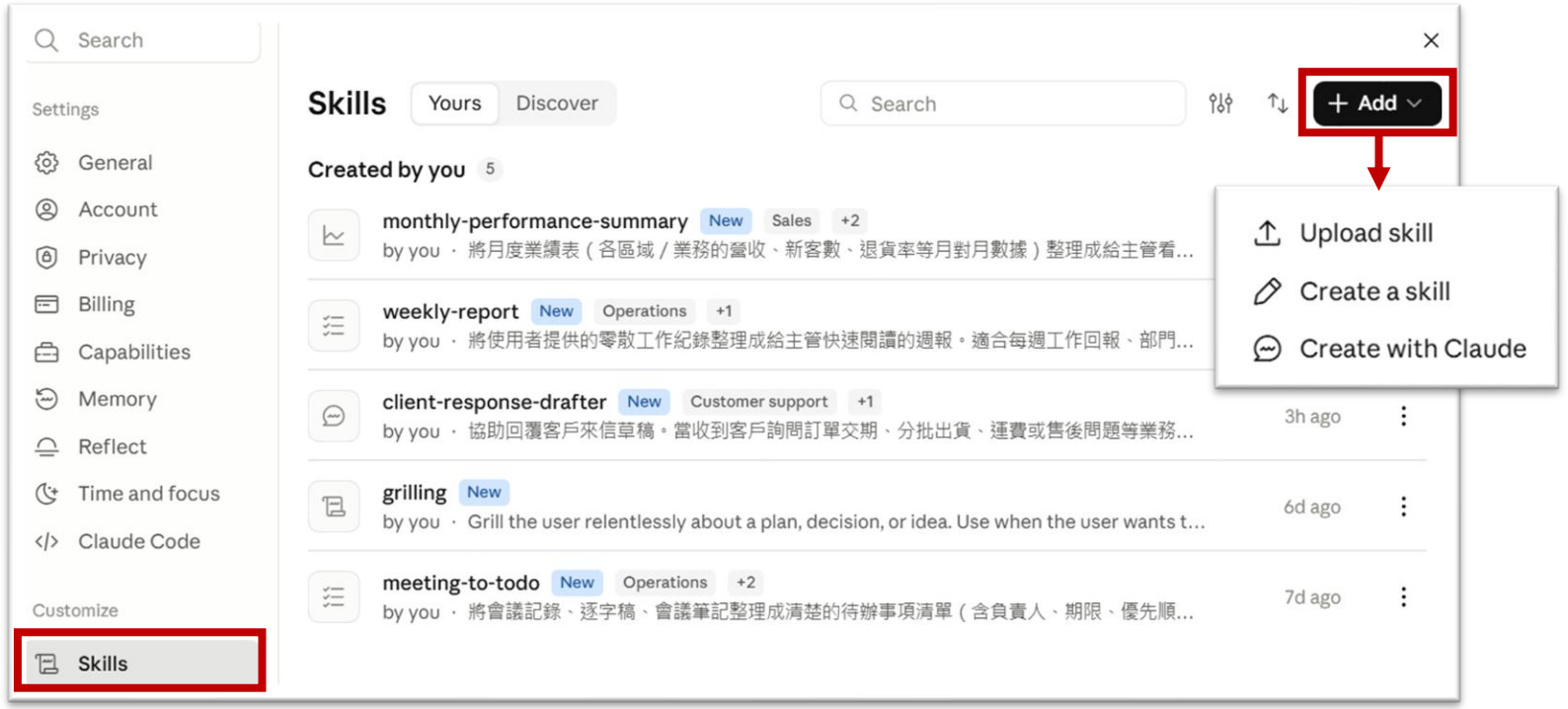
3 使用

直接說「幫我整理會議記錄」即自動觸發

看到「讀取…SKILL.md」＝有用到

開始前先確認

Settings › Capabilities 打開「Code execution and file creation」，否則技能不會運作



找現成的SKILL：先問事、再找源、看三點、試後改

01

先問事

從重複工作出發

- 寫下一件每週都做、步驟固定的工作
- 有明確情境，才找得到對的 Skill

02

再找源

由近到遠三層找

- 官方：Anthropic／OpenAI／Google
- 部門專門技能包
- 網站、清單與搜尋

03

看三點

安裝前先確認

- 來源：官方？近期更新？
- 說明：description 符合需求？
- 安全：有 scripts 先給 IT 審

04

試後改

別人的是起點

- 用不含機密的真實資料試跑
- 用 skill-creator 改成公司版本

找 Skill 就像找外包夥伴：先想清楚要交辦什麼，再看他的來歷、讀他的工作說明，試用一次，最後依公司習慣調教。

GitHub 上最熱門的 10 個 Agent Skills

1 **obra/superpowers** ★ 295k
完整開發方法論：設計 → 規劃 → TDD → 協作

2 **mattpocock/skills** ★ 275k
/grill-me 釐清需求、TDD、除錯、架構改善

3 **anthropics/skills** 官方 ★ 180k
官方：Word／PPT／Excel／PDF、品牌、skill-creator

4 **DietrichGebert/ponytail** ★ 153k
「最懶資深工程師」：先問要不要寫，碼量減 54%

5 **garrytan/gstack** ★ 135k
23 個技能扮演 CEO、設計、QA、發佈的虛擬團隊

6 **nextlevelbuilder/ui-ux-pro-max-skill** ★ 133k
配色、字型、UX 規則，自動產生設計系統

7 **addyosmani/agent-skills** ★ 101k
25 個工程技能：定義 → 規劃 → 建置 → 上線

8 **nexu-io/open-design** ★ 99k
開源 Claude Design 替代品，內建 100+ 技能

9 **tt-a1i/archify** ★ 77k
架構圖、流程圖、時序圖，一句話產出可驗證的圖表

10 **ComposioHQ/awesome-claude-skills** ★ 76k
1,000+ 技能清單，可串接 Gmail、Slack 等 SaaS

資料來源：依 GitHub 星數人工整理 統計日 2026/10/4（星數每日變動）

產業 × 熱門 Skills 地圖

通用辦公 ★ 180k

anthropics/skills

Word、PPT、Excel、PDF 產出，品牌規範與內部溝通

行銷 ★ 53k

coreyhaines31/marketingskills

文案、SEO／AI 搜尋優化、轉換率、A/B 測試

金融投資 ★ 39k

anthropics/financial-services

DCF、可比公司估值、投委會備忘錄、KYC 審查

業務・法務・財會 ★ 26k

anthropics/knowledge-work-plugins

拜訪準備、工單分流、合約審閱、對帳與月結

生醫科研 ★ 47k

K-Dense-AI/scientific-agent-skills

166 個研究技能，串接 100+ 科學資料庫

主管決策・法遵 ★ 27k

alirezarezvani/claude-skills

C-level 顧問角色、ISO／GDPR／SOC 2 法遵

設計・UI ★ 133k

nextlevelbuilder/ui-ux-pro-max-skill

配色、字型搭配、依產業自動產生設計系統

教育訓練 ★ 544

anthropics/k12-teacher-skills

教案設計、分級差異化教學（另見 education-agent-skills）

人資・採購・營運 ★ 229

w95/awesome-claude-corporate-skills

JD 撰寫、面試題組、RFP、供應商評估、SOP

資料來源：依 GitHub 星數人工整理 統計日 2026/10/4（星數每日變動）

蒐集 Agent Skills 的網站

1 skills.sh

skills.sh

Vercel 營運

- 排行榜：總安裝數、24 小時趨勢
- 一行指令安裝：npx skills add

適合：看當下最熱門的 Skills

2 SkillsMP

skillsmp.com

社群獨立平台

- 索引 330 萬+ 個 SKILL.md 檔
- 可依 867 種職業分類瀏覽

適合：依自己職務找範例參考

3 agent-skills.cc

agent-skills.cc

社群目錄

- 收錄 6.3 萬+ 技能
- 可依熱門、最新、安裝數篩選

適合：快速比較同類技能熱度

4 agentskills.io

agentskills.io

開放標準官網

- Anthropic 發起的 Agent Skills 規格
- 快速入門、支援工具清單

適合：學寫法、查哪些工具相容

蒐集 Agent Skills 的 GitHub Repo

1	ComposioHQ/awesome-claude-skills	1,000+ 技能依用途分類：文件、資料、行銷、生產力、App 自動化	★ 76.4k
2	hesreallyhim/awesome-claude-code	Claude Code 資源總整理：Skills、Hooks、斜線指令、工作流	★ 55.0k
3	VoltAgent/awesome-agent-skills	1,000+ 技能，主打官方團隊出品：Vercel、Cloudflare、Sentry、Hugging Face	★ 35.2k
4	anthropics/knowledge-work-plugins	官方：依部門分的外掛——業務、行銷、法務、財務、資料、客服	★ 26.0k
5	GitHub Topic：agent-skills	28,000+ 個標記 agent-skills 的公開專案；搜尋語法見下一頁	

資料來源：GitHub 各 repo 頁面星數，統計日 2026/10/4；金色框＝Anthropic 官方；建議先看官方，再用清單類 repo 擴大搜尋

GitHub 搜尋技巧：5 個常用語法

1	**marketing agent skills**	依產業找：英文產業關鍵字＋agent skills 或 claude skills
2	**topic:agent-skills finance**	只搜標記為 agent-skills 的專案，雜訊最少、最推薦
3	**path:SKILL.md HR**	程式碼搜尋：找真的含 SKILL.md 檔的 repo（需先登入 GitHub）
4	**org:anthropics skills**	只看官方帳號；換成 org:openai、org:google 查其他家
5	**topic:agent-skills stars:>1000**	只看 1,000 星以上；改 pushed:>2026-07-01 可篩近 3 個月有更新

語法來源：GitHub Docs〈Searching for repositories〉〈Understanding GitHub Code Search syntax〉｜搜尋後右上角 Sort 選 Most stars；金色框＝只看官方

Skill 安裝前檢查清單

1 看來源

這是誰做的？

- ☐ 維護者是官方或知名團隊？
- ☐ 最近 3 個月內有更新？
- ☐ 星數、Issues 討論（僅供參考）

2 看說明

它真的適合我嗎？

- ☐ 打開 SKILL.md 讀 description
- ☐ 「做什麼」＋「何時觸發」是否符合
- ☐ 會不會跟已安裝的 Skill 搶觸發

3 看安全

企業最該注意的一關

- ☐ 是否附 scripts（可執行程式）？
- ☐ 是否連外部網址、上傳資料？
- ☐ 企業環境：先交 IT／資安審核
- ☐ 來路不明：不裝

⚠ 收錄 300 萬+ Skills 的 SkillsMP 明白寫著「不認證品質與安全」——社群 Skill 請比照開源程式碼看待。

grill-me／grilling：先把你問清楚，再動手

grill-me/SKILL.md —— 全文只有 7 行

```
---
name: grill-me
description: A relentless interview to
  sharpen a plan or design.
disable-model-invocation: true
---
Call the Skill tool with "grilling".
```

▼ 呼叫另一個 Skill

grilling/SKILL.md —— 30 行，零程式（節錄）

```
---
name: grilling
description: Grill the user relentlessly
  about a plan, decision, or idea. Use
  when the user wants to stress-test
  their thinking, or uses any 'grill'
  trigger phrases.
---
Interview the user relentlessly ...
```

它怎麼運作

1. **畫出決策樹**
   把計畫拆成一個個互相牽動的決定，先決條件沒定的先不問
2. **一輪一輪問**
   每輪只問「現在就能回答」的題目，每題都附上 AI 的建議答案
3. **事實 AI 查，決策你做**
   需要查的事實 AI 自己查，要拍板的決定交給你；全部問完、你確認共識後，AI 才動手

教學點：
排行第 2 的 Skill 只有 30 行文字。grill-me 只是入口，真正的本體是 grilling；在 claude.ai 只要上傳 grilling 就能用。

資料來源：github.com/mattpocock/skills（MIT 授權），skills/productivity/grill-me、grilling

在 Claude 使用 grilling 的五個步驟

1 **取得 Skill**
GitHub：mattpocock/skills → skills/productivity/grilling，打包成 zip

2 **上傳到 Claude**
claude.ai 只需上傳 grilling 即可

3 **下指令觸發**
輸入「grill me：」加上你的計畫（例如一場活動、一個專案）

4 **逐輪回答**
逐題回覆，同意建議可直接說「Q1 照建議」

5 **確認後產出**
問題問完、你確認共識後，再請它整理成完整計畫

對話示意

grill me：明年 Q1 想為企業客戶開一門 AI 內訓課

❓ **Q1 目標學員：主管還是第一線員工？**
➡ 建議：先鎖定中階主管，決策權高

❓ **Q2 形式：實體、線上或混成？**
➡ 建議：線上 2 小時 × 3 堂

❓ **Q3 成效怎麼衡量？**
➡ 建議：課後 30 天追蹤流程導入率

（你回答後，依答案再展開下一輪）

💡 **原文是英文也沒關係：用中文提問，Claude 就用中文追問。SKILL.md 裡「派子代理查資料」的段落是給 Claude Code 用的。**

重點回顧

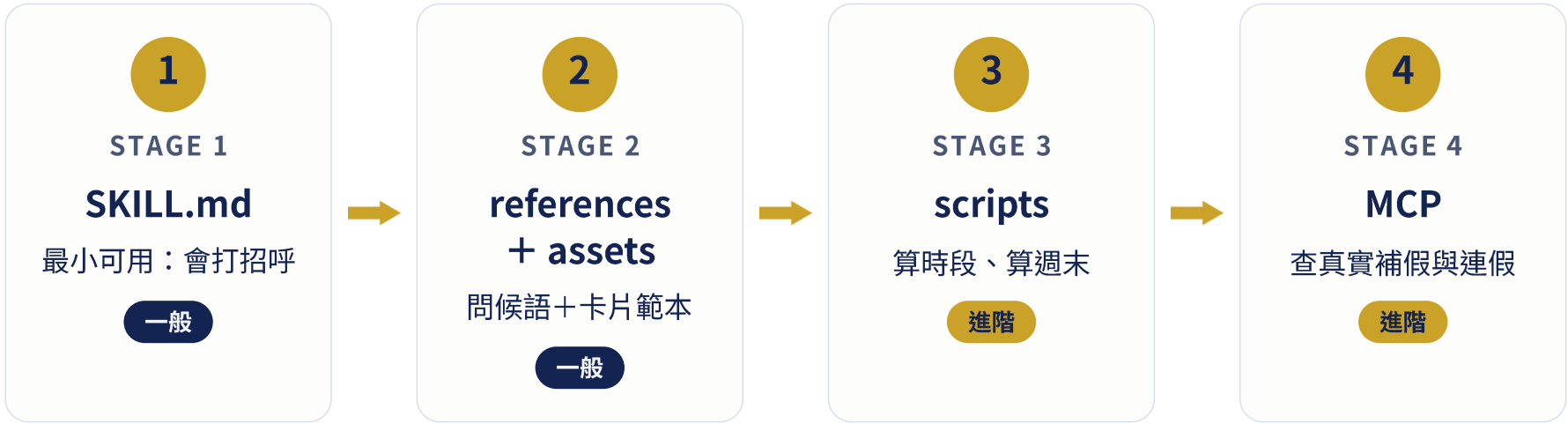
別人的 Skill 是起點，自己的才好用

先問事 › 再找源 › 看三點 › 試後改 › 打造專屬技能包

下一段：動手把自己的工作流程寫成 SKILL.md

一般 進階

一個 hello-skills 的進化之旅



一般使用者做到 Stage 1–2 就很夠用；資訊背景的朋友再往 Stage 3–4 前進。

一般

SKILL.md 須包含兩個部分

YAML frontmatter 上半部・頭部資訊，像填表格

SKILL.md 範例

```
---
name: hello-skills
description: 用活潑方式打招呼。當使用者說「哈囉」時使用。
---
```

1. 冒號用半形「:」，後面**空一格**
2. description **寫成一行**，中間不要按 Enter
3. 上下兩條 **---** 不能少，檔案第一行就是 ---

YAML 官方網站：https://yaml.org/

Markdown content 下半部・指示內容，像寫文件

SKILL.md 範例

```
# Hello Skills
## 流程
1. 先查目前時間
2. 再寫問候語
- 不要自己編問候語
```

1. **#** 大標題、**##** 小標題，# 後空一格
2. **1. 2. 3.** 有順序的步驟
3. **-** 沒順序的規則或條列

Markdown 語法速查表：https://www.markdownguide.org/cheat-sheet/

最常見的三個錯：冒號打成全形「：」、少了一條 ---、description 中間按了 Enter

一般

SKILL.md 長什麼樣

```
hello-skills/
└── SKILL.md
```

SKILL.md

```
---
name: hello-skills                    ← 必須＝資料夾名稱
description: 用活潑的方式跟使用者打招呼並
  自我介紹。當使用者說「哈囉」「打招呼」
  「hello skills」或要求測試 Skill 時
  使用。
---
# Hello Skills
1. 開頭寫「👋 Hello Skills 已啟動！」
2. 用一句話說明你是由 Skill 驅動的
3. 加上今天的日期與時間
4. 結尾寫「這是我的第一個 Skill 🎉」
```

頭部資訊（必填）

※ description 實際上是同一行，這裡為了排版才換行顯示

1 name

英文小寫＋數字＋連字號，**必須和資料夾同名**
不可使用：空格、大寫、底線、中文

2 description

寫清楚「**做什麼＋什麼時候用**」＋觸發說法
Claude 只看這段決定是否啟用，上限 1024 字元

3 指示內容

步驟、格式、規則——像寫給新同事的 SOP
建議 **500 行以內**，較長資料移到 references/

檔名請寫成 **SKILL.md**（習慣用 全大寫），放在與 name 同名的資料夾裡，例如 **hello-skills/SKILL.md**

資料來源：agentskills.io/specification、Claude Docs – Skill authoring best practices

description 寫得好，Skill 才會被叫出來

⚠ 不好的寫法

description: 打招呼工具

→ 太短：沒說何時用，也沒有關鍵字

description: 我可以幫你打招呼

→ 第一人稱：官方建議用第三人稱描述

✓ 好的寫法

name: hello-skills

做什麼	依台北時段用指定語言打招呼，產出問候卡
什麼時候用	說「哈囉」「還有幾天放假」「hello skills」時
什麼時候不用	只是要翻譯一句問候語
需要什麼	想用的語言（沒指定就用繁體中文）

句型公式 直接套用，把【】換成自己的內容

【做什麼】。當使用者【說法 1、2、3（中英都放）】時使用，即使沒有明說也一樣。【不適用：○○】

1. 「何時使用」只寫在 description，內文在觸發前看不到
2. 寫完要實測：該觸發、不該觸發的說法都要測

一般

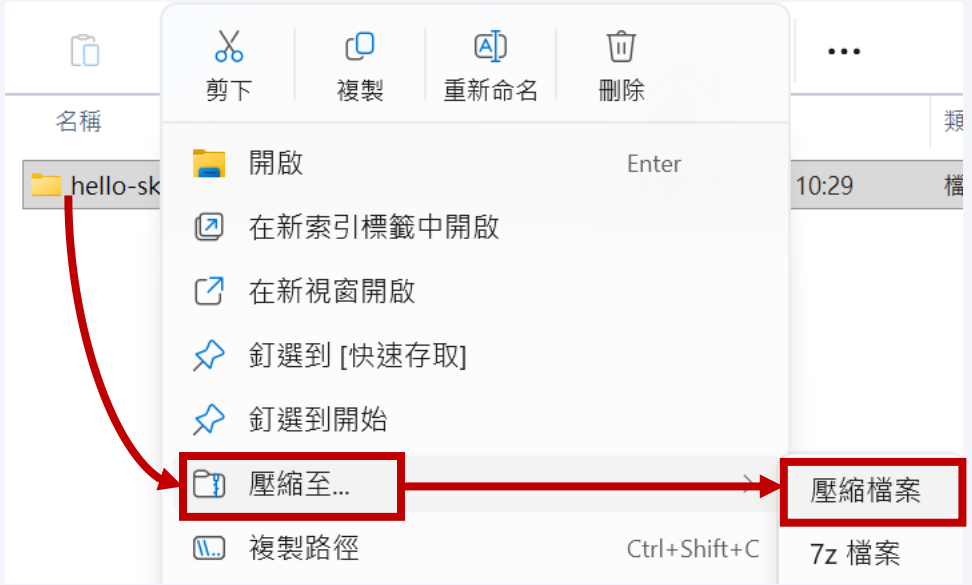
最小可用的 Skill：只要一個 SKILL.md

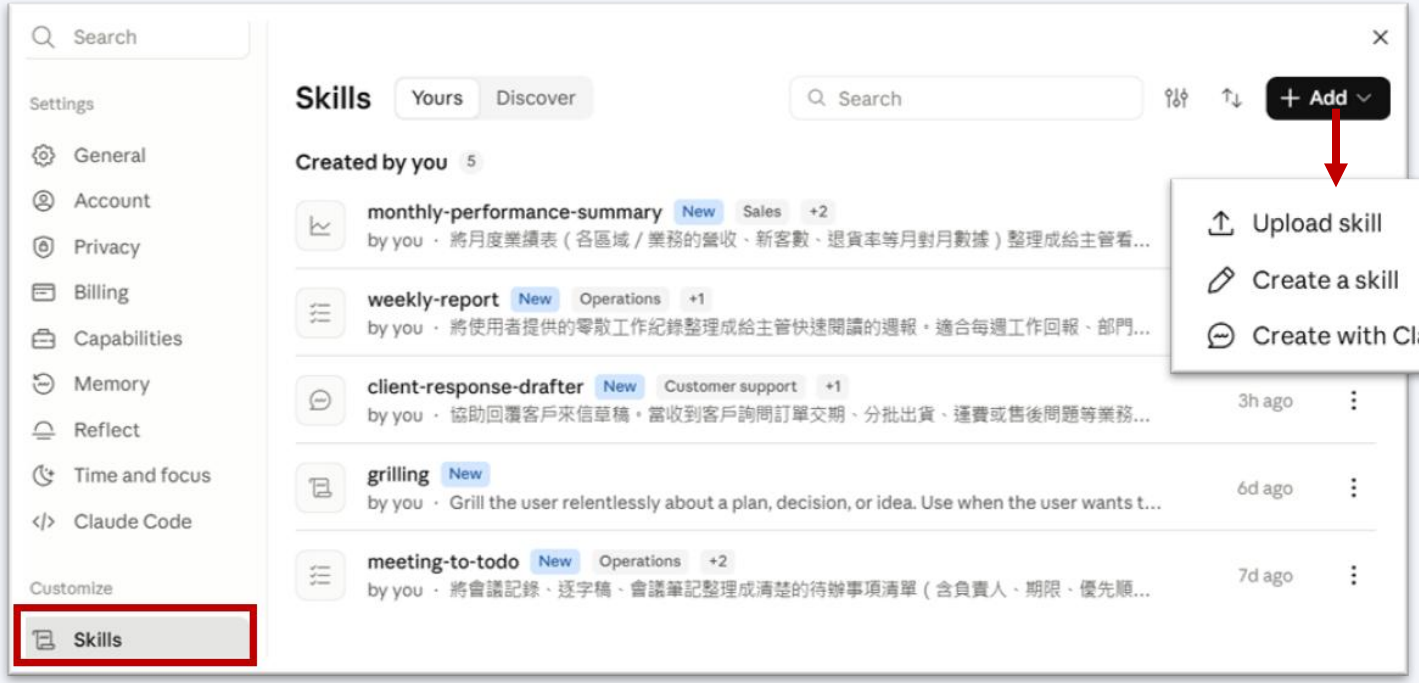
資料夾結構

```
hello-skills/
└─ SKILL.md
```

上傳流程

① 資料夾按右鍵 → 壓縮至 → **壓縮檔案**
② 設定 → Skills → **Upload skill**
③ **開新對話**，用觸發說法測試





常見卡關：選到 7z，或只壓縮了資料夾裡的檔案
沒反應時：檢查設定 → Capabilities 的程式碼執行

一般

資料和範本，分開放

1 references／給 AI「讀」

例：**greetings.md** 各語言問候語

特性：會被**讀進來理解**，所以要短、主題單一

判斷：**要先看懂才能做事** → 像員工手冊

2 assets／給 AI「用」

例：**card-template.md** 問候卡範本

特性：**不必讀懂**，照樣套用或複製

判斷：**會原樣出現在成品裡** → 像公司信紙

資料夾結構

```
hello-skills/
├── SKILL.md
├── references/
│   └── greetings.md
└── assets/
    └── card-template.md
```

→

SKILL.md（片段）

```
## 流程
1. 判斷使用者要的語言
2. 到 references/greetings.md 查問候語
3. 套用 assets/card-template.md 輸出
```

沒在 SKILL.md 寫到的檔案，Claude 不會主動打開

寫法規則：路徑用相對路徑（如 references/greetings.md）；參考檔只放一層，不要層層互相引用 ｜ 資料來源：agentskills.io/specification

分開放的好處：SKILL.md 保持精簡好讀；資料和範本用到才載入；要更新時只改對應的小檔案

進階

用 skill-creator 測試與改良 Skill

1 準備測試句

該觸發、不該觸發的句子各寫幾句

2 對照測試

有 Skill vs 沒 Skill，比較輸出品質

3 優化 description

依測試結果改寫描述，提升觸發準確度

在 Claude 建立 SKILL

用 skill-creator 幫我建立一個 Skill。
我想要 Claude 每次被打招呼時，用固定的格式回應並隨機產生一句勵志語句。
請先問我問題、確認需求，再開始寫 SKILL.md。

在 Claude 測試 SKILL

用 skill-creator 幫我測試 [Skill名稱]：
設計應觸發與不應觸發的測試句，
跑一輪對照測試，
再幫我優化 description。

測試案例就是 Skill 的品質保證：每次改版前後都跑一次。

一般

一份填空模板：今晚就能做出你的 Skill

my-skill/SKILL.md

```
---
name: [英文小寫-用連字號]
description: [這個技能做什麼]。當使用者
 [提到哪些詞／提供什麼資料] 時使用。
---
# [技能名稱]
## 輸入：使用者會提供 [  ]
## 步驟
1. [  ]
2. [  ]
## 輸出格式：[  ]
## 規則
- 資料不足時標註「待確認」，不要自行編造
```

填寫提示

- name：英文小寫與連字號
- description：同時寫「做什麼」與「何時用」
- 規則寫清楚「不知道就標註」
- 寫完 → 打包 ZIP → 上傳 → 測試

Skill 的結構：遵循 agentskills.io 開放標準

SKILL.md	**YAML frontmatter**（name＋description）：定義中繼資料與觸發條件 **Markdown 內文**：工作流程、輸入輸出格式、邊界情況
references/	按需載入的參考知識（Markdown／JSON／YAML）
assets/	靜態資源：HTML 範本、配色方案、資料表
scripts/	可執行程式碼（Python／JS／Bash），處理需要精準計算的步驟

運作機制：Progressive Disclosure 分層載入——先讀 description，需要時才讀全文與附件

① 永遠載入

名稱＋描述

每個 Skill 只佔一兩句話，AI 靠它判斷要不要用

② 觸發時載入

SKILL.md 全文

做事步驟、規則、輸出格式

③ 需要時才開

references／assets／scripts

大量資料不必一開始就讀

一般

YAML frontmatter：有哪些欄位？

欄位	必填	規則	範例
name	**是**	1–64 字元，**小寫英數＋連字號**；不能以 - 開頭結尾或連用 --；**須與資料夾同名**	hello-skills
description	**是**	1–1024 字元；寫**做什麼＋何時用**，並放入觸發關鍵字	依目前時段打招呼並產出問候卡。 當使用者說「哈囉」時使用
license	否	授權名稱，或附帶的授權檔	Apache-2.0、MIT
compatibility	否	最多 500 字元；執行環境需求（產品、套件、網路）	需要 Python 3
metadata	否	自訂鍵值，例如作者、版本	author: iSpan version: "1.0"
allowed-tools	否	預先核准可用的工具清單（**實驗性**，各平台支援不一）	Read Bash(python:*)

入門只要記：兩條 --- 之間填好 name 和 description 就合規；選填欄位在分享或企業部署時再加

資料來源：agentskills.io/specification

進階

精確的事，交給程式

AI 擅長

理解語意、判斷情境、撰寫內容

程式擅長

時間日期、計算、格式轉換——**每次都要一樣**

scripts/now.py

取得台北時間 → **datetime=2026-10-14 20:05**
判斷目前時段 → **period=evening**

SKILL.md（片段）

1. 執行 **python scripts/now.py**
2. 依 period 查 **references/greetings.md**

⚠ **踩坑紀錄：**執行環境是 UTC 時區，沒換成台北時間（+8），晚上 8 點會被當成中午 12 點

1. **何時交給程式？**
   答案只有一個，算錯就出事
2. **SKILL.md 要寫清楚**
   何時執行、結果怎麼用
3. **只有執行結果進對話**
   程式碼本身不佔 Claude 閱讀空間

※ claude.ai 需開啟「程式碼執行」才能跑 scripts（免費版預設開啟） ｜ 資料來源：agentskills.io/specification

一般 進階

四種內容怎麼選？一張表整理

內容	判斷問題	hello-skills 範例	SKILL.md 寫法
SKILL.md	每次都要照做的**流程和規則**？	6 個步驟、4 條規則	主檔，觸發時讀全文
references/	要**先看懂**才能做事？ → 成品的「依據」	greetings.md weekend-messages.md	到 references/... **查**
assets/	會**原樣出現**在成品裡？ → 成品的「外殼」	card-template.md	**套用** assets/⋯
scripts/	**答案只有一個**，算錯就出事？ → 只有執行結果進對話	now.py weekend.py	**執行** python scripts/⋯

分層載入：用到才讀，不浪費 Claude 的閱讀空間

1 **一直都在 name＋description**
每個 Skill 約 100 tokens

→

2 **觸發時 SKILL.md 全文**
流程、規則一次讀進來

→

3 **用到才開 三個子資料夾**
scripts 只有執行結果進對話

原則：SKILL.md 只放一定要知道的，其他放子資料夾；**但每個檔案都要在 SKILL.md 寫到，Claude 才會去開**

進階

Skill 多了，AI 怎麼選？

使用者說的話

↓

比對所有 Skill 的 description

↓

選出最符合的 Skill

↓

讀取 SKILL.md 並執行

一般使用者也能做

- 用不到的 Skill 關掉開關，減少互相搶
- 新增或修改後，用該觸發／不該觸發的說法測試

1 描述重疊就會選錯

canvas-design 與 frontend-design 常互搶

解法 ▸ description 寫清「做什麼＋何時用」，加一句「若是○○，改用 X」

2 「沒看到」≠「沒裝好」

在指令選單找不到，不代表沒裝好；大多數 Skill 靠 description 自動觸發

解法 ▸ 到「設定 → Skills」確認開關；用 description 裡寫的說法實際測試

3 企業上百個 Skill：先縮小範圍再選

目錄太長，description 彼此干擾也更佔空間 → **100 個 → 5 個候選 → 1 次呼叫**

① 規則篩選
依部門、角色過濾

→

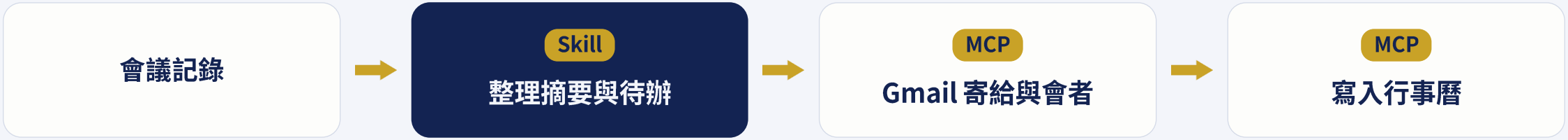
② 語意比對
找出最像的幾個

→

③ LLM 精挑
在候選中選 1 個

進階

Skill 搭配 MCP：從「會想」到「會做」



Skill 負責「怎麼想」

整理方式、格式、規則

MCP 負責「去哪查、怎麼做」

查資料、寄信、寫行事曆：連到 Gmail、日曆、內部系統

進階

Skill 是 SOP，MCP 是外線電話

一句話

Skill 是 SOP 手冊，
MCP 是外線電話。

Claude 照 SOP 做事；SOP 寫著「這一步打電話問」，Claude 就撥那支電話。

角色	比喻	在 Demo 裡
Claude	**新來的行政助理**	聽懂需求、照流程做事
Skill	**主管給的 SOP 手冊**	hello-skills-mcp：步驟與範本
scripts	**桌上的計算機**	now.py：看現在幾點、什麼時段
MCP	**打去人事室的電話**	hello-mcp：查辦公日曆（補假、連假）
退路	**電話不通，先看桌曆**	weekend.py，標示「⚠️ 未含國定假日」

計算機算得出今天星期幾，卻不知道今年哪天補假——會變動的外部資料，就打電話去問。

進階

MCP 的兩種接法：用現成的，或自己做

Demo 1 現成連接器

例子	Gmail、Google 日曆
誰開發	服務商（例如 Google）
跑在哪	雲端
怎麼接	設定 › 連接器 › 按「連接」，登入帳號授權（OAuth）
哪裡能用	網頁版、Desktop、手機都能用
適合	通用服務，連上就能用

Demo 2 自建 MCP

例子	hello-mcp（查政府辦公日曆）
誰開發	自己或公司 IT
跑在哪	自己電腦或公司主機
怎麼接	寫設定檔，指定用哪個程式啟動 Server
哪裡能用	本機版：Desktop、Claude Code；網頁版要部署成公開 HTTPS
適合	公司內部系統、專屬資料

有現成的就用現成的；要接公司內部系統、專屬資料，才自己做。

進階

Demo 1：會議記錄 Skill × Gmail × Google 日曆

1
匯入逐字稿
「整理這個會議紀錄」

→

2 Skill
Skill 整理
摘要＋待辦表，人名查對照表

→

3 MCP
Gmail 建草稿
只建立草稿，還不寄出

→

4
你確認
看過收件人與全文，回覆「寄出」

→

5 MCP
寄出＋排行事曆
寄出信件，寫入 Google 日曆

SKILL.md 裡寫好的三條安全規則

1 先草稿、再確認、才寄出

寄信不能收回；你沒說「寄出」，就只停在草稿。

2 查不到就說，不捏造

對照表找不到 Email 的人，直接告訴你需要另外提供。

3 沒日期的不排行事曆

「待定」「盡快」的項目另外列出，請你確認後再排。

進階

Demo 2：自己做一個 MCP——hello-mcp

問題

weekend.py 只看「星期幾」

不知道國定假日、補假、補班。10/9（五）是國慶日補假，它卻說「明天才放假」。

解法

會變動的資料，交給 MCP 去查

`get_next_day_off(date)`



Skill 管流程、scripts 管計算、MCP 管外部資料——各做自己擅長的事。

進階

Demo 2：把 hello-mcp 接上 Claude Desktop

① 建立 venv 並安裝（PowerShell）

```
cd hello-mcp
python -m venv .venv
.venv\Scripts\pip install mcp
# 預先下載今年辦公日曆
.venv\Scripts\python server.py `
  --prefetch 2026
```

② 寫進 claude_desktop_config.json

```
{
 "mcpServers": {
  "hello-mcp": {
   "command": "...\\.venv\\Scripts\\python.exe",
   "args": ["...\\hello-mcp\\server.py"]
  }
 }
}
```

③ 重開並確認

完全結束 Desktop（含系統匣）再打開

設定 › 開發者，看到 hello-mcp 正在執行

問「還有幾天放假？」

最常卡關

command 要寫 venv 裡的 python.exe。Desktop 不會幫你 activate，寫 python 會出現 No module named 'mcp'。

哪裡能用

本機 MCP 只在 Claude Desktop、Claude Code 有效；網頁版在雲端，要部署成公開 HTTPS 才連得到。

進階

Demo 2：同一句話，三種結果

示範提示詞 **假設今天是 10/9（五），還有幾天放假？**

1 只有 Skill

原版 hello-skills

🔥 明天就放假了！再撐一天，週末在向你招手～

× 錯：10/9 是國慶日補假

2 Skill＋hello-mcp

hello-skills-mcp＋連上 hello-mcp

今天放假！（補假、國慶日）連假 10/09（五）–10/11（日）

✓對：查到真實辦公日曆

3 拔掉 MCP

hello-skills-mcp，沒有 hello-mcp

距離週末還有 1 天（10/10 星期六）⚠️ 未含國定假日與補班

✓不當機，誠實標示

進階

Skill＋MCP

1

Skill 管流程，MCP 管外部資料與動作

會變動的資料、需要權限的動作交給 MCP，其他留在 Skill。

2

一定要寫退路

連接器可能斷線；寫好 B 計畫，並誠實標示資料不完整。

3

安全規則寫在 Skill

MCP 收到呼叫就執行；「先給我看過再做」只能寫在 SKILL.md。

怎麼選？

有現成連接器
→ 直接連接，例如 Gmail

要接公司內部系統
→ 自建 MCP

網頁版也要能用
→ 部署成公開 HTTPS 服務

同一套做法，換成 Slack、Notion、公司 ERP 都適用。

進階

企業整合：透過 API 使用 Skills

你的系統
ASP.NET、Java Spring、Python …

↓

Messages API
＋ Code Execution ＋ Skills

↓

回傳文字與產出檔案

1 claude.ai 上傳的 Skill 不能直接給 API 用

需在 Console 或用 Skills API（ant apply）另外上傳，取得 skill_id

2 container.skills 是授權清單

不是執行命令；是否使用仍由 Claude 依描述判斷

3 每次請求最多 20 個 Skill

單一任務頁面只放 1 個，行為最穩定

```
container={"skills": [{"type": "custom", "skill_id": skill_id, "version": "latest"}]},
```

資料來源：Claude Platform Docs — Using Agent Skills with the API

進階

網頁 → FastAPI → Claude API + MCP

1

網頁

index.html

選語言與日期、確認貼出、看執行軌跡

2

FastAPI 後端

app.py

保管 API Key、當 MCP Host 代呼叫工具

3

Messages API

skill_id＋code_execution

在容器裡讀 SKILL.md、跑 now.py、查 references

4

hello-mcp

stdio（本機）

get_next_day_off 查放假日、post_card 貼留言板

為什麼由後端代呼叫 MCP？

API 內建的 MCP connector 要公開 HTTPS 網址；由後端代呼叫，本機 localhost 的 Server 就能用。

API Key 只放在後端

網頁只呼叫自己的 FastAPI，瀏覽器永遠拿不到 Key，也不能直接打 Claude API。

進階

Messages API：用程式跟 Claude 對話

POST /v1/messages | 一次請求的組成

```
client.messages.create(
  model="claude-opus-5-5",    # 選模型
  max_tokens=4096,          # 回答長度上限
  messages=[...],           # 對話內容
  container={"skills": [...]},# 授權 Skill
  tools=[code_execution,    # 開啟程式執行
       *mcp_tools],         # MCP 工具定義
)
```

① 沒有記憶

每次呼叫都獨立；多輪對話要把前面的訊息一起再傳。

② 能力靠參數開啟

Skill、程式執行、網路搜尋，都是同一支 API 加參數。

③ 回傳一串內容區塊

tool_use → 後端代呼叫工具；pause_turn → 接著續跑。

三方分工
Skills API 上架（ant apply） › Messages API 使用（skill_id＋工具） › MCP 提供工具（後端代呼叫）

進階

MCP 工具：後端當 Host，代 Claude 呼叫

唯讀：get_next_day_off

查政府辦公日曆，回傳 days_left、假日名稱、連假天數；Claude 要用時，後端直接呼叫。

寫入：post_card

把卡片寫進留言板（outbox.md）；使用者在網頁按「確認貼出」後，後端才放行。

步驟	MCP Python SDK
啟動 Server	stdio_client(params)
初始化	session.initialize()
列出工具	session.list_tools()
轉成 tools	{name, description, input_schema}
呼叫工具	session.call_tool(name, args)

使用前要知道

每次請求啟動一次：stdio 開子行程約 1 秒，demo 夠用

工具說明原封不動：description 交給 Claude 判斷何時使用

寫入動作兩道關卡：Skill 規定先問，後端再擋一次

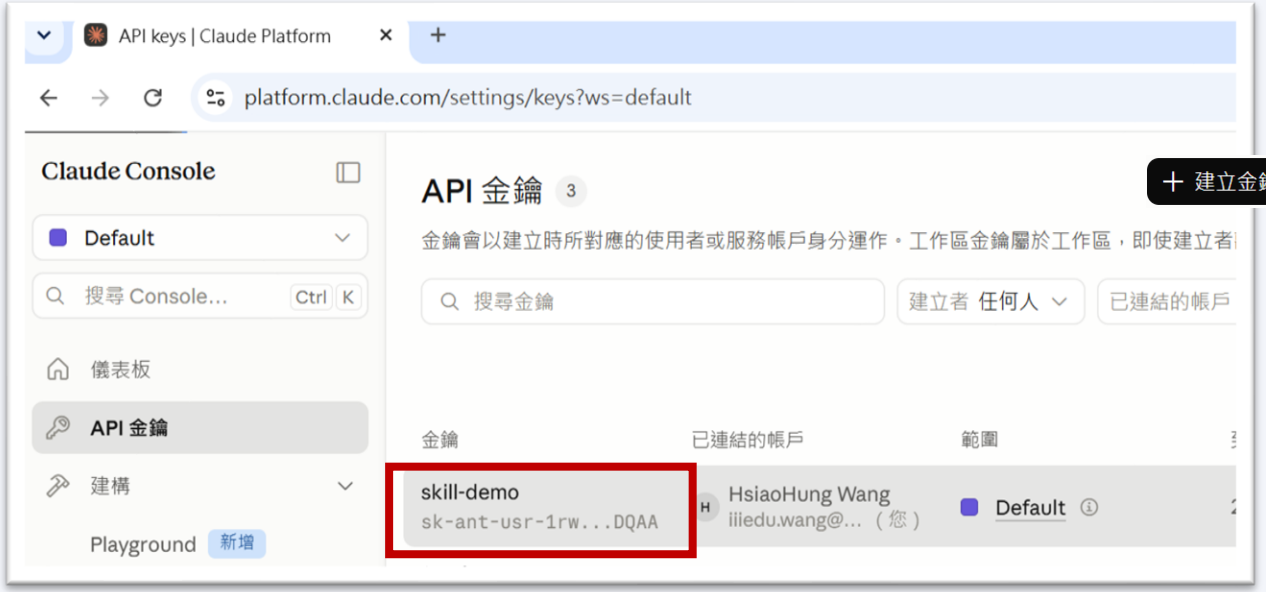
資料來源：skill-demo-1014\hello-web\app.py、MCP Python SDK

進階

在 Claude Console 建立 API Key

API 金鑰 › + 建立金鑰

欄位	設定	說明
Name	skill-demo	一個專案一把，方便停用
Expires	30 天	demo 用，到期自動失效
Linked account	你本人	正式上線改綁服務帳號
Scope	Default workspace	Skill 也會上傳到這裡



PowerShell | 把 API 存在 Windows 的環境變數中

```
setx ANTHROPIC_API_KEY "sk-ant-..."
# 重開終端機後生效
```

Python | 讀取 Windows 的環境變數中的 ANTHROPIC_API_KEY
ANTHROPIC_API_KEY

```
from anthropic import AsyncAnthropic
client = AsyncAnthropic()
```

Key 只顯示一次：沒記下來只能重建一把

Workspace 要一致：上傳和呼叫 Skill 用同一個 workspace 的 Key

不要寫進程式或前端：也不要上傳到 GitHub

進階

ant：Anthropic 官方的 API 命令列工具

ant 是什麼

把 Claude API 包成指令，不寫程式也能上傳 Skill、傳訊息、管理檔案。

ant ≠ Claude Code

Claude Code 讀本機 skills 資料夾，不會上傳到 API，也不會產生 skill_id。

計費方式

走 Console 的 API 額度，跟 Claude Pro／Team 訂閱分開計算。

PowerShell | Windows 安裝步驟

```
winget install GoLang.Go      # 1. 安裝 Go（1.22 以上），裝完重開視窗
go install github.com/anthropics/anthropic-cli/cmd/ant@latest  # 2. 安裝 ant
ant --version                 # 3. 確認安裝成功
ant auth status               #確認用哪把 Key、哪個 Workspace
```

出現「找不到 ant」？把 %USERPROFILE%\go\bin 加進 PATH，重開視窗再試。Mac 可直接 brew install anthropics/tap/ant。

進階

ant apply 一行指令，拿到 SKILL_ID

資料夾結構（解壓縮後）

```
skill-demo-1014\
├─ skills\          ← 在這層執行
|  ├─ claude-lock.json
|  └─ hello-skills-mcp\  ← 上傳這層
|     ├─ SKILL.md
|     └─ assets\ references\ scripts\
├─ hello-mcp\       ← MCP Server
└─ hello-web\       ← FastAPI＋網頁
```

PowerShell

```
cd C:\workspace\1_claude\AgentSkills\skill-demo-1014\skills
ant apply hello-skills-mcp
Apply these changes? y
+ ./hello-skills-mcp  created
  skill_01EXn2ZVk58rFsDtkFMGVY7H
```

claude-lock.json

記錄 skill_id 和版本；app.py 啟動時自動讀取，不用手動貼 SKILL_ID。

① 是資料夾，不是 zip

② SKILL.md 要在第一層

③ 在 Skill 的上一層執行

進階

用 test.py 確認 API 叫得到 Skill

test.py（節錄）

```
lock = json.load(open("../skills/claude-lock.json"))
skill_id = lock["resources"]["./hello-skills-mcp"]["id"]
resp = client.messages.create(
  model="claude-opus-5-5", max_tokens=4096,
  container={"skills": [{"type": "custom",
    "skill_id": skill_id, "version": "latest"}]},
  tools=[{"type": "code_execution_20250825",
    "name": "code_execution"}],
  messages=[{"role": "user", "content":
    "哈囉！假設今天是 2026-10-14，幫我做一張問候卡。"}])
```

先建虛擬環境

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python test.py
```

怎麼確認 Skill 有作用

- 依台北時段自動用「午安」
- 沒接 MCP → 走 weekend.py 並標 ⚠️
- 鼓勵語取自 references（駝峰日快樂）

進階

app.py：多當一個 MCP Host，把 Skill 包成服務

端點	做什麼
POST /api/greet	收名字、語言、日期 → 啟動 hello-mcp → 帶 skill_id＋工具呼叫 Claude
POST /api/confirm	使用者按「確認貼出」→ 同一段對話繼續，放行 post_card
GET /api/status ・ outbox ・ /	狀態列（SKILL_ID、MCP 工具）・ 留言板 outbox.md ・ 網頁

① 工具清單從 MCP 來

list_tools() 的名稱、說明、參數格式，轉成 Messages API 的 tools。

② tool_use 迴圈

Claude 要用工具就停下，後端呼叫 MCP、送回 tool_result 再續跑。

③ 寫入要人確認

沒按確認就呼叫 post_card，後端回錯誤不執行，軌跡顯示 ⛔。

啟動後端

```
pip install -r requirements.txt ; uvicorn app:app --port 8000   # Windows 不加 --reload；開啟 http://localhost:8000
```

進階

網頁只做一件事：呼叫自己的 FastAPI

static/index.html（節錄）

```
const res = await fetch('/api/greet', {
  method: 'POST',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({
    name, language, as_of,
    use_mcp, post_to })
});
const data = await res.json();
// data.text  → 問候卡框＋說明文字
// data.awaiting_confirm → 顯示「確認貼出」
// data.trace → 執行軌跡（Skill／MCP）
```

1. **輸入：**名字、語言、日期、MCP 開關、貼給誰
2. **確認：**Claude 先秀卡片，按確認才寫入
3. **輸出：**問候卡、執行軌跡、留言板

小技巧

取消勾選 hello-mcp，就能現場示範退路：卡片改標 ⚠。

進階

改規則只動 Skill，前後端都不用改

① 修改 Skill

改 SKILL.md、references 或 scripts

② ant apply

顯示 update；skill_id 不變、版本 ID 更新

③ 直接生效

呼叫用 "version": "latest"，下一次就是新版

實例：連假鼓勵語

修改前：只有一般放假句子

10/9 補假連假，只顯示「週末快樂」。

修改後：新增 long-weekend 區塊

references 加兩句，連假自動顯示「共 3 天」。

實際踩到的坑

改完忘了 ant apply

本機檔案是新版，API 上還是舊版，結果照舊。

怎麼確認

看 claude-lock.json 的版本 ID 有沒有改變。

進階

卡住時，先對照這張表

症狀	原因與解法
網頁顯示未設定 SKILL_ID	還沒 ant apply → 在 skills 資料夾執行，產生 claude-lock.json
hello-mcp 未連線	hello-mcp\.venv 沒建好 → 照 README 建 venv、安裝 mcp
有接 MCP 卻標 ⚠️	辦公日曆下載失敗 → 先跑 server.py --prefetch 2026
Skill 沒作用	沒帶 code_execution 工具 → tools 加上 code_execution_20250825
NotImplementedError	Windows 加了 --reload → 拿掉 --reload 再啟動
API 回 skill 不存在	Key 跟 Skill 不在同一個 workspace → 用同一個 workspace 的 Key

一句話：Skill 管流程、MCP 管外部資料與寫入；Key 和 MCP 都在後端，網頁只呼叫自己的 FastAPI。

一般

企業導入：從一個任務開始

1 挑一個任務

什麼樣的任務最適合？

- 每週都在做、步驟固定
- 做錯了代價低、容易檢查
- 例：會議紀錄、週報、客訴分類

2 試用並量化

用數字說服老闆

- 小組試用 2 週
- 記錄每次省下的時間
- 收集同事回饋，修正 Skill

3 建立 Skill 庫

從一個部門到全公司

- 好用的放進共用目錄
- 每個 Skill 指定負責人維護
- 再複製到其他部門

效益怎麼算？

每週次數 × 每次省下分鐘 × 使用人數 = 每週省下的時間

例：會議紀錄 Skill
5 場會議 × 省 20 分鐘 × 30 人

≈ 每週 **50** 小時

公司不能用 Claude？

- Skill 是 agentskills.io 開放格式
- 資料夾可帶到公司核准的平台
- 先在個人環境做好，再交 IT 評估

💡 先讓一個部門用得起來，有了省時數字，再談全公司佈署。

一般

企業導入：治理與安全的四件事

1 資料分級

最該先定的規則

- 不可給：客戶個資、病歷、未公開財報
- 可給：已去識別化或公開的資料
- 選企業方案或 API：資料不用於模型訓練

2 來源與權限

把 Skill 當成軟體看待

- 只用自製或官方 Skill（見安裝前檢查清單）
- allowed-tools 限制可用工具（Claude Code）
- Enterprise 方案可開啟內容掃描

3 人工覆核

AI 產出，由人負責

- 重要文件由人簽核後才送出
- 把規則寫進 Skill：「查無資料寫『待確認』」
- 保留輸入與輸出紀錄，方便稽核

4 版本與佈署

Skill 就是資料夾＋文字檔

- 放進 Git，追蹤每次修改
- Team／Enterprise 由管理者統一佈署、指定部門
- 個人 Pro 方案需每人自行上傳

⚠ **金融業可對照金管會「金融業運用人工智慧（AI）指引」，導入前先和法遵、資安單位確認。**

一般

會後下一步：依你的角色開始

1 行政與職場工作者

本週挑一件重複工作

用 skill-creator 做成第一個 Skill

和同事分享資料夾

2 資訊背景

加入 references／scripts

串接 MCP，寫測試案例

嘗試透過 API 整合

3 主管與人資

盤點部門 SOP

建立內部 Skill 庫

訂出審核與佈建流程

從一個小 Skill 開始，比規劃一套大系統更快看到成效。

THANK YOU

謝謝參與
Q&A 時間

規格　agentskills.io

官方　github.com/anthropics/skills

教學　support.claude.com

